LAB = 3

Q1 W.A.P Bubble Sort in Ascending Order

```
#include <iostream>
Using namespace Std;
int main(){
int a[] = {5,3,8,2,1};
int n=5, temp;

Cout<<" Before Souting :";
for( int i=0;i<n; i++)
    Cout<< a[i]<<" ";
    Cout<<endl;

for( int i=0; i<n-1; i++){
    for( int j=0; j<n-i-1; j++){
      if( a[j]> a[j+1]){
            temp= a[j];
            a[j]= a[j+1];
          a[j+1]= temp;
}
}
    Cout<<" Pass "<< i+1<<" : ";
for( int K=0; K<n; K++)
        Cout << a[K] <<" ";
}  Cout<< endl;
```

return 0;

}

Sample O/P :-

Before Sorting : 53821

Pass 1 : 35218

Pass 2 : 32158

Pass 3 : 21358

Pass 4 : 12358

W.A.P Bubble Sort in Descending Order :-

```
#include <iostream>
Using namespace std;
int main()<
int a[] = <5, 3, 8, 2, 1};
int n = 5, temp;
for( int i=0; i<n-1; i++)<
for( int j=0; j<n-i-1; j++)<
if( a[j] < a[j+1])<
temp = a[j];
a[j] = a[j+1];
a[j+1] = temp;
}
}
cout<< " Sorted array : ";
for( int i=0; i<n; i++)
cout<< a[i]<< " ";
return 0;
}
```

Sample O/P :-

Sorted Array : 85321

W.A.P Insertion Sort in Ascending Order.

```
#include <iostream>
Using namespace std;
int main(){
    int a[] = {5,3,8,2,1};
    int n=5, key, j;
    for( int i= 1 ; i<n ; i++){
        key = a[i];
        j = i-1;
    while( j>=0 && a[j] > key){
        a[j+1] = a[j];
        j--;
    }
    a[j+1] = key;
    Cout<<"Pass "<<i<<": ";
    for( int k = 0; k<n; k++)
        Cout<<a[k]<<" ";
    Cout<<endl;
}
}
```

Sample O/P :-

Pass ; 35821

Result :- 12358

W.A.P Selection Sort in Ascending Order

```
#include <iostream>
Using namespace std;
int main(){
int a[] = {5,3,8,2,1};
int n=5, min, temp;
for( int i=0; i<n-1; i++){
    min = i;
for( int j= i+1 ; j<n ; j++){
        if ( a[j] < a[min])
            min = j;
}
cout<<" Pass " << i+1 << minimum = " << a[mi
    <<endl;
temp = a[i];
a[i]= a[min];
a[min]= temp;
}
Cout<< " Sorted array :";
for( int i=0; i<n ; i++){
        cout << a[i] << " ";
return 0;
}
```

Sample O/P:-

Sorted Array : 1 2 3 5 8

W.A.P : Merge Sort

```
#include <iostream>
using namespace std;

void merge( int a[], int l, int m, int h){
  int temp[100], i=l, j=m+1, k=0;
  while( i<=m && j<=h){
    if(a[i]<a[j])
      temp[k++]=a[i++];
    else
      temp[k++]=a[j++];
  }
  while(i<=m)
    temp[k++]=a[i++];

  while(j<=h)
    temp[k++]=a[j++];

  for(i=l, k=0; i<=h; i++, k++)
    a[i]=temp[k];
}

void mergesort(int a[], int l, int h){
  if(l<h){
    int m=(l+h)/2;
    mergesort(a, l, m);
```

W.A.P : Merge Sort

```
#include <iostream>
using namespace std;

void merge( int a[], int l, int m, int h){
  int temp[100], i=l, j=m+1, k=0;
  while( i<=m && j<=h){
    if(a[i]<a[j])
      temp[k++]=a[i++];
    else
      temp[k++]=a[j++];
  }
  while(i<=m)
    temp[k++]=a[i++];

  while(j<=h)
    temp[k++]=a[j++];

  for(i=l, k=0; i<=h; i++, k++)
    a[i]=temp[k];
}

void mergesort(int a[], int l, int h){
  if(l<h){
    int m=(l+h)/2;
    mergesort(a, l, m);
```

W.A.P : Merge Sort

```
#include <iostream>
Using namespace std;

Void merge( int a[], int l, int m, int h){
  int temp[100], i=1, j=m+1, k=0;
  while( i<=m && j<=h){
    if(a[i]<a[j])
      temp[k++]=a[i++];
    Else
      temp[k++]=a[j++];
  } while( i<=m)
      temp[k++]=a[i++];

  While( j<=h)
      temp[k++]=a[j++];

  for( i=1, k=0; i<=h; i++; k++)
      a[i]=temp[k];

}
  Void merge sort( int a[], int l, int h){
    if(l<h){
      int m=(l+h)/2;
    merge sort(a, l, m);
```

```
    merge(a, l, m, h);
  }
}

int main(){
  int a[] = {40, 20, 50, 10, 30};
  int n = 5;

  mergeSort(a, 0, n-1);

  cout<<"Sorted array";
  for(int i=0; i<n; i++){
    cout<<a[i]<<" ";
  }

  return 0;
}
```

Sample O/P :-

Sorted Array : 10 20 30 40 50

W.A.P Quick Sort :-

```
#include<iostream>
Using namespace std;

Void quickSort( int a[], int low, int high){
    if( low >= high) return ;

  int pivot = a[high];
    int i= low;
  for( int j= low; j< high; j++){
    if ( a[j] < pivot){
        Swap (a[i], a[j]);
          i++;
    }
  }
    Swap (a[i], a[high]);
    quickSort ( a, low, i-1);
    quickSort ( a, i+1, high);
}
  int main(){
    int a[] = {5,3,8,2,1};
    int n=5;
    quickSort(a,0,n-1);
    cout<< " Sorted array :";
    for( int i=0; i<n; i++)
        cout<< a[i]<<" ";
```

return 0;

}

Output :-

Sort Array : 1 2 3 5 8

W.A.P Quick Sort after Each partition

```
#include<iostream>
Using namespace std;
int a[5] = {5, 3,8,2,1};
int partition (int low, int high){
    int pivot = a[high], i=low;
for( int j= low; j<high; j++){
    if (a[j]< pivot){
        Swap(a[i], a[j]);
        i++;
    }
}
    Swap(a[i], a[high]);
Cout<<" Partition ";
for( int k=0; k<5; k++){
    Cout<< a[k]<<" ";
Cout<<endl;
return i;
}
Void quickSort(int low, int high){
    if( low < high){
int p = partition(low, high);
quickSort(low, p-1);
```

```
    quickSort( pt+1, high);
}
int main(){
    quickSort(0,4);
    cout<<"Sorted array:";
for(int i=0; i<5; i++){
    cout<<a[i]<<" ";
}
    return 0;
}
```

Sample Output:-

Sorted array: 1 2 3 5 8

W.A.P

W.A.P Compare Three Sorting Algorithms

```
#include <iostream>
Using namespace std;
Void bubblesort(int a[], int n, int &c, int &s)
{   for(int i=0; i<n-1; i++)
    for(int j=0; j<n-i-1; j++){
        c++;
        if(a[j]>a[j+1]){
            Swap(a[j], a[j+1]);
            s++;
        }
    }
}
Void insertion(int a[], int n, int &c,
                int &s){
    for(int i=1; i<n; i++){
        int key = a[i], j=i-1;
        while(j>=0){
            c++;
            if(a[j]>key){
                a[j+1]=a[j];
                s++;
                j--;
```

```
    } else break;
  } a[j+1] = key;
 }
}

void selection(int a[], int n, int &c, int &s){
    for( int i=0; i<n-1; i++){
        int min = i;
    for( int j= i+1; j<n; j++){
        c++;
    if( (a[j]<a[min]) min = j;
    }
    if( min != i){
    swap(a[i], a[min]);
    s++;
    }
  }
}

int main(){
    int a[] = {5, 3, 8, 2, 1};
    int b[] = {5, 3, 8, 2, 1};
    int d[] = {5, 3, 8, 2, 1};
    int c1 = 0, s1 = 0, c2 = 0, s2 = 0,
        c3 = 0, s3 = 0;
```

```
bubble(a, 5, c1, s1);
insertion(b, 5, c2, s2);
Selection(d, 5, c3, s3);

Cout << "Bubble Sort : Comparisons = " << c1
     << ", Swaps = " << s1 << endl;
Cout << "Insertion Sort : Comparisons = " << c2
     << ", Shifts = " << s2 << endl;
Cout << "Selection Sort : Comparisons = " << c3
     << ", Swaps = " << s3 << endl;

return 0;
}
```

Sample O/P :-

Bubble Sort : Comparision = 10, Swaps = 8

Insertion Sort : Comparision = 9, Shifts = 7

Selection Sort : Comparision = 10, Swaps = 3

W.A.P Merge Sort & QuickSort

```
#include <iostream>
Using namespace std;
Void merge( int a[], int l, int m, int h){
int t[5], i=1, j=m+1, k=0;
while( i<=m && j<=h)
  t[k++] = (a[i] < a[j]) ? a[i++] : a[j++];
while( i<=m) t[k++] = a[i++];
while( j<=h) t[k++] = a[j++];
for( i=1, k=0, i<=h; i++, k++)
    a[i] = t[k];
}
Void mergeSort( int a[], int l, int h){
    if( l<h){
    int m = (l+h)/2;
    mergeSort( a, l, m);
    mergeSort( a, m+1, h);
    merge( a, l, m, h);
}
Void quickSort( int a[], int l, int h){
```

```
if( l >= h) return ;
int pivot = a[h], i = l;
for( int j = l ; j < h; j++)
    if( a[j] < pivot)
        Swap(a[i++], a[j]);
Swap(a[i], a[h]);
quickSort(a, l, h-1);
quickSort(a, i+1, h);
}
int main(){
    int a[] = { 5, 3, 8, 2, 1};
    int b[] = {5, 3, 8, 2, 1};
    mergeSort( a, 0, 4};
    quickSort( b, 0, 4);
cout << " Merge Sort : ";
for( int x: a) cout << x << " ";
cout << "In Quick Sort :";
for( int x : b) cout << x << " ";
return 0;
}
```

W.A.P Menu - Driven Sorting :-

```
#include <iostream>
Using namespace std;

int main(){
 int a[5]={5,3,8,2,1};
 int ch, temp;

 Cout<<" 1, Bubble Sort In 2 - Selection
                    Sort In";
 Cout<<" 3. Insertion Sort In";
 Cout<<" Enter Choice : ";
 Cin>>ch;

 if( ch==1 || ch== 2 || ch == 3 ){
   for( int i=0; i<4; i++ ){
     for( int j=0 ; j<4-i ; j++ ){
       if((ch==1 && a[j] > a[j+1])
     || (ch==2 && a[j] > a[j+1]) ||
       (ch==3 && a[j]> a[j+1])){
         temp= a[j];
```

```
} else {
        cout << " Invalid choice ! ";
        return 0;
    }
    cout << " Sorted array : " ;
    for ( int i=0; i<5; i++ )
        cout << a[i] << "  ";
    return 0;
}
```

Sample O/P :-

1. Bubble Sort
2. Selection Sort
3. Insertion Sort

Enter Choice : 1

Sorted array : 1 2 3 5 8

W.A.P Count Comparisons in Three Sorting algorithms.

```
#include <iostream>
Using namespace std;
int main() {
    int a[] = {5,3,8,2,1};
    int n=5, b[5], c[5], x=0, y=0,
        z=0;
    for( int i=0; i<n; i++){
        b[i] = c[i] = a[i];

for( int i=0; i<n-1; i++)
    for( int j=0; j<n-i; j++){
        x++;
        if( a[j] > a[j+1])
            Swap( a[j], a[j+1]);
    }
    for( int i=1; i<n; i++){
        int key = b[i], j=i-1;
        while( j>=0){
            y++;
            if( b[j] > key){
                b[j+1] = b[j];
                j--;
    }
```

```
        else break;
    }
    b[j+1] = key;
}

for( int i=0; i<n-1; i++){
    int min = i;
    for( int j= i+1; j<n; j++){
        z++;
        if( c[j] < c[min] )
            min = j;
    }
    swap(c[i], c[min]);
}

cout<<"Bubble Sort Comparision : "<<u<<endl;
cout<<"Insertion Sort Comparision : "<<y<<endl;
cout<<"Selection Sort Comparision : "<<z<<endl;

    return 0;
}
```

W.A.P Best Case of Bubble Sort & Insertion Sort

```
#include <iostream>
Using namespace Std;

int main(){

    int a[] = { 1,2,3,4,5};
    int n = 5, c1 = 0, c2 = 0;

for( int i=0; i<n-1; i++){
    for( int j=0; j<n-i-1; j++){
        c1++;
    if( a[j] > a[j+1] )
        Swap(a[j], a[j+1]);
    }
}

    for( int i=1; i<n; i++){
        int Key = a[i], j=i-1;
        while( j>=0){
            c2++;
        if( a[j]>Key ){
            a[j+1] = a[j];
            j--;
        } else break;

}
```

```
        a[j+1] = key;
    }
    cout<<"Bubble Sort Comparision : "<<c1<<endl;
    cout<<"Insertion Sort Comparision : "<<c2<<endl;
}
```

Sample O/P :-

```
Bubble Sort Comparision : 10
Insertion Sort Comparision : 4
```

W.A.P Test Quick Sort on different Arrays :-

```
#include <iostream>
using namespace std;

int partition(int a[], int low, int high){
    int pivot = a[high], i = low;
    for(int j = low; j < high; j++)
        if(a[j] < pivot)
            swap(a[i++], a[j]);
    swap(a[i], a[high]);
    return i;
}

void quickSort(int a[], int low, int high){
    if(low < high){
        int p = partition(a, low, high);
        quickSort(a, low, p-1);
        quickSort(a, p+1, high);
    }
}

void display(int a[], int n){
    for(int i=0; i<n; i++)
        cout << a[i] << " ";
    cout << endl;
}
```

```
int main(){
int a[] = {1,2,3,4,5};
int b[] = {5,4,3,2,1};
int c[] = {8,1,5,2,4};
  int n=5;

  quickSort(a, 0, n-1);
  quickSort(b, 0, n-1);
  quickSort(c, 0, n-1);

  Cout << "Sorted array: ";
  display(a,n);

  Cout << "Reverse - Sorted array ";
  display(b,n);

  Cout << "Reverse array: ";
  display(c,n);

  Return 0;
}
```

Sample O/P :-

Sorted Array : 1 2 3 4 5

Reverse - Sorted array : 1 2 3 4 5

Random array : 1 2 3 4 5